

Élaboration d'un protocole ZKP basé sur le schéma de Diffie–Hellman pour la 2FA et l'échange de clé sécurisé dans un environnement PKI hiérarchique

Mémoire de Master 2



Contents

Chapter 1

Introduction générale

1.1 Contexte

Présenter les mécanismes classiques d'authentification dans les systèmes distribués (mot de passe, certificats, OTP). Expliquer le rôle des infrastructures à clés publiques (PKI) hiérarchiques dans l'authentification moderne.

1.2 Problématique

Montrer les limites des schémas existants :

- exposition des secrets lors de l'authentification,
- dépendance à des facteurs uniques,
- absence de garanties cryptographiques formelles pour certains mécanismes 2FA.

Formuler explicitement la problématique centrale :

Comment concevoir un protocole d'authentification forte à deux facteurs reposant sur Diffie–Hellman, permettant un échange de clé sécurisé, tout en préservant la confidentialité des secrets grâce aux preuves à divulgation nulle de connaissance, dans un environnement PKI hiérarchique ?

1.3 Objectifs du mémoire

Lister clairement :

- concevoir un protocole ZKP basé sur Diffie–Hellman,
- intégrer un mécanisme réel de 2FA,
- garantir l'échange de clé sécurisé,
- analyser formellement la sécurité du protocole,
- proposer une implémentation prototype.

1.4 Contributions

Énumérer les contributions attendues sans survente :

- conception d'un protocole original combinant ZKP, 2FA et PKI,
- analyse de sécurité qualitative,
- validation expérimentale par prototype.

Chapter 2

Fondements cryptographiques

2.1 Groupes cycliques et hypothèse du logarithme

Définir formellement :

- **Groupe cyclique** (G, g, q) :
 - G est un groupe d'ordre premier q ,
 - $g \in G$ est un générateur : tout élément $h \in G$ peut s'écrire $h = g^x$ pour un certain $x \in \mathbb{Z}_q$,
 - Notation et propriétés : associativité, élément neutre, inverses.
- **Hypothèse du logarithme discret (DLP)** :
 - Problème : étant donné $g, h \in G$, trouver x tel que $h = g^x$,
 - DLP est considéré difficile dans des groupes correctement choisis (par ex. $\mathbb{Z}_p^*$ ou groupes elliptiques),
 - Cette hypothèse est la base de la sécurité des protocoles DH et Schnorr.

Ajouter un exemple concret avec petit p pour illustration pédagogique.

2.2 Échange de clé Diffie–Hellman

Présenter le schéma classique :

- Chaque partie choisit un secret : $a, b \xleftarrow{\$} \mathbb{Z}_q$,
- Calcul des messages publics : $A = g^a, B = g^b$,
- Clé partagée : $K = g^{ab}$.

Discuter des propriétés :

- **Sécurité** : dériver K sans a ou b revient au problème DH,
- **Confidentialité** : K reste secret même sur un canal public,
- **Résistance aux attaques passives et actives** (mentionner MitM si non authentifié).

Illustrer éventuellement avec un petit diagramme d'échange.

2.3 Preuves à divulgation nulle de connaissance

Définir rigoureusement :

- **Complétude** : un prouveur honnête peut convaincre le vérificateur honnête,
- **Solidité** : un prouveur malhonnête ne peut convaincre le vérificateur qu'avec une probabilité négligeable,
- **Divulgence nulle** : le vérificateur n'apprend aucune information supplémentaire sur le secret au-delà de sa validité.

Ajouter un petit exemple abstrait (comme "caverne des couleurs") pour intuition pédagogique.

2.4 Preuve de Schnorr

Présenter le protocole comme un ZKP de connaissance du logarithme discret :

- Paramètres : (G, g, q) , secret x , élément public $Y = g^x$,
- Étapes :
 1. Engagement : $r \xleftarrow{\$} \mathbb{Z}_q$, $R = g^r$ envoyé au vérificateur,
 2. Défi : le vérificateur envoie $c \xleftarrow{\$} \mathbb{Z}_q$,
 3. Réponse : $s = r + cx \pmod{q}$, le prouveur envoie s ,
 4. Vérification : $g^s \stackrel{?}{=} R \cdot Y^c$.
- Souligner : base pour prouver la connaissance de x sans la révéler,
- Mentionner propriétés : complétude, solidité, divulgation nulle,
- Lien avec le protocole proposé : utilisé pour vérifier le facteur 1 (clé privée PKI).

Ajouter éventuellement un diagramme d'interaction (prover $\rightarrow$ verifier) pour visualisation.

Chapter 3

Modèle système et hypothèses

3.1 Environnement PKI hiérarchique

Dans ce travail, l'authentification repose sur une infrastructure à clés publiques hiérarchique. Il convient de préciser les éléments suivants :

- **Autorité de certification racine (Root CA) :**
 - détient une clé privée fortement protégée,
 - signe les certificats des autorités intermédiaires,
 - sert de point de confiance ultime pour la vérification des certificats.
- **Autorités de certification intermédiaires (Intermediate CAs) :**
 - reçoivent leur certificat de la Root CA,
 - émettent et gèrent les certificats des utilisateurs finaux,
 - permettent la délégation de confiance tout en limitant l'exposition de la clé racine.
- **Certificats utilisateurs :**
 - contiennent la clé publique $Y = g^x$ de l'utilisateur et son identité,
 - signés par une CA intermédiaire, validant la possession de la clé privée x ,
 - utilisés pour vérifier l'authenticité de l'utilisateur dans le protocole.

Il est utile d'inclure un diagramme hiérarchique montrant la Root CA, les Intermediate CAs et les utilisateurs.

3.2 Acteurs

Le système considère les acteurs suivants :

- **Utilisateur légitime :**
 - possède une clé privée x certifiée par la PKI,
 - détient un secret secondaire s pour le second facteur 2FA,
 - participe activement au protocole pour prouver son identité.
- **Serveur d'authentification :**
 - vérifie les preuves à divulgation nulle de connaissance,

- génère des challenges et la clé de session Diffie–Hellman,
- maintient la sécurité des échanges et des certificats.
- **Autorité de certification :**
 - émet et révoque les certificats utilisateurs,
 - établit la chaîne de confiance hiérarchique,
 - joue un rôle passif pendant les sessions d’authentification.

3.3 Modèle de menace

L’analyse de sécurité considère différents types d’attaquants et scénarios de compromission :

- **Attaquant réseau passif :**
 - peut intercepter les messages entre l’utilisateur et le serveur,
 - ne peut pas modifier ou injecter de nouvelles communications,
 - objectif : récupérer des informations sensibles comme la clé de session ou les preuves ZKP.
- **Attaquant réseau actif :**
 - peut intercepter, modifier, supprimer ou injecter des messages,
 - objectif : tenter un attaque de type Man-in-the-Middle (MitM) ou usurpation d’identité,
 - le protocole doit résister à ces attaques grâce aux ZKP et à l’authentification forte.
- **Compromission d’un seul facteur :**
 - hypothèse : l’attaquant obtient soit la clé privée x , soit le secret secondaire s ,
 - analyse la sécurité du protocole dans ces conditions,
 - objectif : démontrer que la possession d’un seul facteur ne suffit pas pour s’authentifier.

Il est recommandé d’ajouter un tableau synthétique ou un diagramme d’attaques pour visualiser les scénarios et les facteurs compromis.

Chapter 4

Protocole Sigma-2FA : Authentification Mutuelle Forte à Deux Facteurs avec Confidentialité d'Identité

4.1 Spécification technique

Version	1.0 (2026)
Statut	Proposition de standard
Basé sur	SIGMA-I, TLS 1.3
Garanties	PFS, confidentialité d'identité, authentification 2FA

Table 4.1: Caractéristiques du protocole

4.2 Fonctions cryptographiques requises

4.2.1 Fonctions standard

- $\text{DDH-secure}(G, g, q)$: groupe où DDH est difficile
- $\text{H2S}(msg) \rightarrow \mathbb{Z}_q$: Hash-to-Scalar (RFC 9380)
- $\text{HKDF}(salt, IKM, label, context)$: HKDF (RFC 5869)
- $\text{AEAD-Enc}(K, nonce, plaintext, ad)$: Chiffrement authentifié
- $\text{EdDSA-Sign}(sk, msg)$: Signature EdDSA (RFC 8032)
- $\text{ECDH}(sk, PK)$: Échange de clés éphémère

4.2.2 Constantes de contexte

```
1 CONTEXT_STRINGS = {  
2     "handshake": b"sigma-2fa-handshake",  
3     "encryption": b"sigma-2fa-encryption",  
4     "authentication": b"sigma-2fa-auth",  
5     "application": b"sigma-2fa-application",  
6     "client-fin": b"sigma-2fa-client-finish",
```

```

7     "server-fin": b"sigma-2fa-server-finish",
8     "challenge": b"sigma-2fa-challenge"
9 }

```

4.3 Protection contre le déni de service (DoS)

4.3.1 Mécanisme de cookie

Pour atténuer les attaques par inondation avant le calcul de $B = g^b$, le serveur peut implémenter un mécanisme de cookie optionnel :

1. **Client** $\rightarrow$ **Serveur** : $A = g^a$
2. **Serveur** $\rightarrow$ **Client** : cookie = $\text{HMAC}_{K_S}(A \parallel \text{timestamp})$
3. **Client** $\rightarrow$ **Serveur** : (A, cookie)
4. **Serveur** : Vérifie le cookie avant de calculer B

```

1 def generate_cookie(A, timestamp, server_key):
2     """G n re un cookie pour att nuer les DoS"""
3     return HMAC(server_key, A || timestamp)[0:16]
4
5 def verify_cookie(A, cookie, timestamp, server_key, window
6     =2):
7     """V rifie un cookie avec une fen tre temporelle"""
8     if abs(current_time() - timestamp) > window:
9         return False
10    expected = generate_cookie(A, timestamp, server_key)
11    return constant_time_compare(cookie, expected)

```

4.4 Schéma de nonces AEAD sécurisé

Pour éviter toute réutilisation de nonce même en cas d'erreur d'implémentation :

```

1 class NonceGenerator:
2     def __init__(self, base_nonce):
3         """base_nonce: 96 bits unique la session"""
4         self.base = base_nonce
5         self.counter = 0
6
7     def next(self):
8         """G n re le prochain nonce par XOR avec le
9         compteur"""

```

```

9      # XOR des 12 derniers octets avec le compteur
10     nonce = bytearray(self.base)
11     for i in range(12):
12         nonce[-(i+1)] ^= (self.counter >> (8*i)) & 0xFF
13     self.counter += 1
14     return bytes(nonce)

```

Initialisation : $\text{base_nonce} = H_{96}(K_{\text{handshake}} \parallel \text{"nonce"})$

4.5 Protocole principal

4.5.1 Phase 0 : Pré-négociation (optionnelle)

1. Client envoie ClientHello avec :
 - Version du protocole
 - Cipher suites supportés
 - Extension 2FA-Support indiquant la méthode d'authentification
 - Demande de cookie (optionnel)
2. Serveur répond avec ServerHello incluant :
 - Version sélectionnée
 - Cipher suite
 - Cookie si demandé
 - nonces_S (128 bits)

4.5.2 Phase 1 : Échange de clés éphémère

Étape	Client	Serveur
1	$a \leftarrow \mathbb{Z}_q, A = g^a$	Reçoit A
2		$b \leftarrow \mathbb{Z}_q, B = g^b, \text{sid} = H(A \parallel B)$
3	Reçoit $(B, \text{nonces}_S, \text{sid})$	
4	$K_{\text{DH}} = B^a$	$K_{\text{DH}} = A^b$
5	$T_1 = \text{sid} \parallel \text{nonces}_S$	$T_1 = \text{sid} \parallel \text{nonces}_S$

Table 4.2: Phase 1: Échange Diffie-Hellman

4.5.3 Dérivation des clés de handshake

$$\begin{aligned}
 K_{\text{handshake}} &= \text{HKDF}(\emptyset, K_{\text{DH}}, \text{"handshake"}, T_1) \\
 K_{\text{enc}} &= \text{HKDF}(\emptyset, K_{\text{handshake}}, \text{"encryption"}, \text{sid}) \\
 K_{\text{base_nonce}} &= H_{96}(K_{\text{handshake}} \parallel \text{"nonce-base"}) \\
 \text{nonce}_C &= \text{NonceGenerator}(K_{\text{base_nonce}})
 \end{aligned}$$

4.5.4 Phase 2 : Authentification du client (chiffrée)

Calcul de la preuve ZKP agrégée

$$\begin{aligned}
 r_1, r_2 &\xleftarrow{\$} \mathbb{Z}_q \\
 R_1 &= g^{r_1}, R_2 = g^{r_2} \\
 c &= \text{H2S}(A \parallel B \parallel \text{nonce}_S \parallel R_1 \parallel R_2 \parallel \text{"challenge"}) \\
 z_1 &= r_1 + c \cdot x_U \mod q \\
 z_2 &= r_2 + c \cdot s \mod q
 \end{aligned}$$

Construction du message

```

1 client_auth_message = {
2     "type": 0x01, # CLIENT_AUTH
3     "id": ID_U, # Identifiant opaque
4     "R1": R1,
5     "R2": R2,
6     "z1": z1,
7     "z2": z2,
8     "timestamp": current_time()
9 }
```

Chiffrement :

$$\text{auth}_C = \text{AEAD-Enc}(K_{\text{enc}}, \text{nonce}_C.\text{next}(), \text{client_auth_message}, T_1)$$

Vérification côté serveur

Algorithm 1 Vérification de l'authentification client

Entrée: $\text{auth}_C, T_1, K_{\text{enc}}$

Sortie: Accepté ou rejeté

- 1: $\text{msg} \leftarrow \text{AEAD-Dec}(K_{\text{enc}}, \text{auth}_C, T_1)$
 - 2: Vérifier $\text{msg.type} = 0x01$
 - 3: Récupérer (Y_U, PK_2) depuis la BD via msg.id
 - 4: Recalculer $c = \text{H2S}(A \| B \| \text{nonce}_S \| R_1 \| R_2 \| \text{"challenge"})$
 - 5: Vérifier $g^{z_1} \stackrel{?}{=} R_1 \cdot Y_U^c$
 - 6: Vérifier $g^{z_2} \stackrel{?}{=} R_2 \cdot PK_2^c$
 - 7: Vérifier timestamp dans la fenêtre ± 2 min
 - 8: $T_2 \leftarrow T_1 \| \text{auth}_C$
-

4.5.5 Phase 3 : Authentification du serveur et confirmation de clé

Étape	Serveur	Client
1	$\sigma_S = \text{Sign}_{x_S}(\text{"server-auth"} \ T_2)$	
2	$\text{fin}_S = \text{HKDF}(\emptyset, K_{\text{handshake}}, \text{"server-fin"}, T_2)[0 : 16]$	
3	$\text{auth}_S = \text{AEAD-Enc}(K_{\text{enc}}, \text{nonce}_C.\text{next}(), \sigma_S \ \text{fin}_S, T_2)$	Reçoit auth_S
4		Vérifie σ_S , recalcule et vérifie fin_S
5		$\text{fin}_C = \text{HKDF}(\emptyset, K_{\text{handshake}}, \text{"client-fin"}, T_2)[0 : 16]$
6	Reçoit confirm_C	$\text{confirm}_C = \text{AEAD-Enc}(K_{\text{enc}}, \text{nonce}_S.\text{next}(), \text{fin}_C \ \text{auth}_S, T_2)$
7	Vérifie fin_C	

Table 4.3: Phase 3: Authentification mutuelle

4.6 Clés de session applicatives

Après confirmation mutuelle, les clés finales sont dérivées :

$$\begin{aligned}
 K_{\text{app}} &= \text{HKDF}(\emptyset, K_{\text{handshake}}, \text{"application"}, T_2 \| \text{auth}_S \| \text{confirm}_C) \\
 \text{sid}_{\text{final}} &= H_{256}(T_2 \| \text{auth}_S \| \text{confirm}_C) \\
 K_{\text{rekey}} &= \text{HKDF}(\emptyset, K_{\text{handshake}}, \text{"rekeying"}, \text{sid}_{\text{final}})
 \end{aligned}$$

4.6.1 Politique de renouvellement de clé

- Renouveler après 2^{64} octets chiffrés
- Renouveler après 24 heures
- Utiliser K_{rekey} pour dériver de nouvelles clés sans nouveau handshake

4.7 Gestion du facteur 2

4.7.1 Enrôlement sécurisé

Algorithm 2 Enrôlement du facteur secondaire

- 1: **Entrée** : Utilisateur authentifié avec facteur 1
 - 2: Serveur génère $\text{nonce}_{\text{enroll}} \xleftarrow{\$} \{0, 1\}^{128}$
 - 3: Client génère $s \leftarrow \mathbb{Z}_q$, $PK_2 = g^s$ sur dispositif sécurisé
 - 4: Client signe : $\sigma_{\text{enroll}} = \text{Sign}_{x_U}("2\text{FA-enroll"} \parallel ID_U \parallel PK_2 \parallel \text{nonce}_{\text{enroll}})$
 - 5: Serveur vérifie la signature et enregistre $(ID_U, PK_2, \text{timestamp})$
 - 6: **Sortie** : Facteur 2 enregistré avec preuve de possession
-

4.7.2 Récupération de compte

En cas de perte du dispositif facteur 2, le protocole de récupération doit être au moins aussi fort que l'authentification initiale :

1. Authentification via facteur 1 + canal hors bande (SMS, email de secours)
2. Période de quarantaine (24-48h) avant réactivation
3. Notification sur tous les dispositifs enregistrés
4. Invalidation de toutes les sessions existantes

4.8 Preuve de sécurité informelle

4.8.1 Sécurité de la preuve ZKP agrégée

Théorème 4.8.1: S

us l'hypothèse DLP dans G et dans le modèle de l'oracle aléatoire, la preuve (z_1, z_2) constitue une preuve à divulgation nulle de connaissance de (x_U, s) .

Esquisse de preuve :

1. **Complétude** : Si le prouveur possède (x_U, s) , les vérifications $g^{z_1} = R_1 \cdot Y_U^c$ et $g^{z_2} = R_2 \cdot PK_2^c$ réussissent par construction.
2. **Spécial honest-verifier ZK** : Un simulateur peut choisir c, z_1, z_2 aléatoires, puis calculer $R_1 = g^{z_1} \cdot Y_U^{-c}$, $R_2 = g^{z_2} \cdot PK_2^{-c}$.
3. **Extracteur** : En réexécutant avec deux défis différents c, c' , on peut extraire :

$$x_U = \frac{z_1 - z'_1}{c - c'} \mod q, \quad s = \frac{z_2 - z'_2}{c - c'} \mod q$$

4.8.2 Résistance aux attaques par relais

Théorème 4.8.2: L

protocole résiste aux attaques Man-in-the-Middle grâce au channel binding dans le défi c .

Preuve : Le défi c dépend de A et B . Un attaquant tentant de relayer les messages entre client et serveur aura des valeurs $A' \neq A$ ou $B' \neq B$, menant à un défi $c' \neq c$ et donc à l'échec de la vérification.

4.9 Considérations d'implémentation

4.9.1 Validation des entrées

```

1 def validate_public_key(P):
2     """Valide qu'un point appartient au groupe"""
3     if P == identity_element:
4         return False
5     if not is_on_curve(P):
6         return False
7     if not is_in_subgroup(P):
8         return False
9     return True
10
11 def validate_scalar(z):
12     """Valide qu'un scalaire est dans [0, q-1]"""
13     return 0 <= z < q
    
```

4.9.2 Gestion des erreurs

- Messages d'erreur génériques : "Authentication failed"

- Journalisation côté serveur uniquement
- Limite de tentatives : 5 par minute par IP
- Timeouts : 30 secondes pour chaque phase

4.9.3 Compatibilité

Standard	Compatibilité
TLS 1.3	Peut être implémenté comme extension
FIDO2	Facteur 2 compatible avec WebAuthn
OAuth 2.0	Peut être utilisé comme grant type

Table 4.4: Compatibilité avec les standards existants

4.10 Performance

Opération	Client	Serveur
Exponentiations	4	4
Signatures	0	1
Vérifications de signature	1	0
Opérations AEAD	3	3
Allers-retours	2-3	2-3

Table 4.5: Complexité cryptographique

Conclusion

Le protocole Sigma-2FA atteint les objectifs suivants :

1. **Authentication forte** : Nécessite deux facteurs indépendants
2. **Confidentialité persistante** : Forward secrecy via DH éphémère
3. **Confidentialité d'identité** : Identités protégées contre la surveillance
4. **Résistance au DoS** : Mécanismes d'atténuation optionnels
5. **Authentication mutuelle** : Client et serveur authentifiés
6. **Standardisable** : Compatible avec les infrastructures existantes

Ce protocole est prêt pour l'implémentation et la standardisation. Une preuve formelle complète dans le modèle UC serait l'étape suivante pour une adoption large.

Prochaines étapes

1. Implémentation de référence en Rust/Go
2. Audit de sécurité indépendant
3. Proposition de RFC à l'IETF
4. Intégration avec les navigateurs web et systèmes d'exploitation

Derniers raffinements pour la perfection

Pour que ce document soit "imperméable", voici deux détails mineurs à considérer :

1. Inclusion des clés publiques dans le défi c : Il est souvent recommandé d'inclure Y_U et PK_2 à l'intérieur du hachage pour c . Bien que le transcript T_1 lie déjà la session, inclure les clés publiques dans le défi empêche certaines attaques théoriques de "substitution de clé" (Key Substitution Attacks). Modification suggérée : $c = \text{H2S}(A\|B\|\text{nonce}_S\|R_1\|R_2\|Y_U\|PK_2\|\text{"challenge"})$.
2. Nonce AEAD : Ton NonceGenerator par XOR est exactement ce que préconise la RFC 8446 (TLS 1.3). C'est le standard d'or pour la sécurité.

Chapter 5

Analyse de sécurité formelle

5.1 Cadre théorique et modèle d'attaque

5.1.1 Modèle de sécurité

Nous analysons le protocole Sigma-2FA dans le modèle du challenger-adversaire standard pour les protocoles d'authentification et d'échange de clés. L'adversaire $\mathcal{A}$ est supposé polynomialement borné et contrôle entièrement le réseau (attaquant actif). Il peut :

- Intercepter, modifier, retarder, réinjecter ou bloquer tout message,
- Initier de nouvelles sessions avec n'importe quelle partie,
- Compromettre certaines clés à long terme selon des scénarios prédéfinis.

5.1.2 Hypothèses cryptographiques

L'analyse repose sur les hypothèses suivantes :

- **DDH** : Le problème de Décision Diffie-Hellman est difficile dans G ,
- **DLP** : Le problème du Logarithme Discret est difficile dans G ,
- **ROM** : H et H_2S se comportent comme des oracles aléatoires,
- **EUFCMA** : Le schéma de signature utilisé est existentiellement non falsifiable,
- **IND-CCA** : Le schéma AEAD offre l'indistinguabilité sous attaque à chiffré choisi.

5.2 Authentification mutuelle forte

5.2.1 Preuve de connaissance des deux facteurs

Théorème 5.2.1: Authentification du client

Sous les hypothèses DLP et ROM, un adversaire $\mathcal{A}$ ne peut réussir l'authentification client sans posséder simultanément x_U et s , sauf avec probabilité négligeable.

[Esquisse de preuve] Considérons la preuve ZKP agrégée (R_1, R_2, z_1, z_2) .

1. **Extraction du facteur 1** : Par la sécurité de Schnorr, si $\mathcal{A}$ peut produire une réponse z_1 valide pour un défi c donné, alors il existe un extracteur qui peut obtenir x_U avec probabilité non négligeable. Formellement, en réexécutant $\mathcal{A}$ avec deux défis différents c, c' sur le même engagement R_1 , on peut extraire :

$$x_U = (z_1 - z'_1)(c - c')^{-1} \mod q$$

2. **Extraction du facteur 2** : Le même argument s'applique pour extraire s à partir de z_2 :

$$s = (z_2 - z'_2)(c - c')^{-1} \mod q$$

3. **Liaison des preuves** : Le défi $c = \text{H2S}(A \| B \| \text{nonces}_S \| R_1 \| R_2 \| \text{"challenge"})$ lie les deux preuves. Un adversaire ne peut pas réutiliser une preuve valide pour un facteur sans l'autre, car c dépend des engagements R_1 et R_2 simultanément.

4. **Conclusion** : Produire une preuve valide nécessite la connaissance de (x_U, s) , sous peine de violer l'hypothèse DLP.

5.2.2 Authentification du serveur

Théorème 5.2.2: Authentification du serveur

Sous l'hypothèse EUF-CMA pour le schéma de signature, $\mathcal{A}$ ne peut pas se faire passer pour le serveur légitime sans posséder x_S .

La vérification par le client de $\sigma_S = \text{Sign}_{x_S}(\text{"server-auth"} \| T_2)$ garantit que : 1. Le message a été signé par le détenteur de x_S , 2. La signature couvre l'intégralité du transcript T_2 , empêchant toute modification a posteriori, 3. L'inclusion du label "server-auth" prévient les attaques par réflexion.

5.2.3 Channel binding et résistance au MitM

Liaison au canal channel-binding Le défi c lie cryptographiquement l'authentification à la session Diffie-Hellman spécifique.

Pour une session donnée, le triplet (A, B, nonces_S) est unique avec probabilité écrasante. Un adversaire tentant une attaque par relais (MitM) entre deux sessions distinctes obtiendrait :

- Session 1 : $(A_1, B_1, \text{nonces}_{S1}) \rightarrow c_1$
- Session 2 : $(A_2, B_2, \text{nonces}_{S2}) \rightarrow c_2$

Même si $\mathcal{A}$ relaie les messages, $c_1 \neq c_2$ sauf collision négligeable dans H2S. Ainsi, la preuve (z_1, z_2) calculée avec c_1 sera invalide pour c_2 .

5.3 Résistance à la compromission partielle

5.3.1 Compromission d'un seul facteur client

Théorème 5.3.1: Résistance à la compromission partielle

La compromission isolée de x_U ou s ne permet pas à $\mathcal{A}$ de s'authentifier comme le client légitime.

Considérons deux cas :

Cas 1 : x_U compromis, s sécurisé

- $\mathcal{A}$ peut calculer $z_1 = r_1 + c \cdot x_U$ pour tout r_1 ,
- Mais pour calculer $z_2 = r_2 + c \cdot s$, $\mathcal{A}$ a besoin de s ,
- Sans s , $\mathcal{A}$ doit deviner z_2 tel que $g^{z_2} = R_2 \cdot PK_2^c$,
- Cela revient à résoudre DLP pour PK_2 , hypothèse difficile.

Cas 2 : s compromis, x_U sécurisé

- Symétriquement, $\mathcal{A}$ peut calculer z_2 mais pas z_1 ,
- La vérification $g^{z_1} = R_1 \cdot Y_U^c$ échoue sans x_U .

5.3.2 Confidentialité persistante (Perfect Forward Secrecy)

Théorème 5.3.2: Forward Secrecy

La compromission des clés long terme (x_U, s, x_S) ne compromet pas la confidentialité des sessions passées.

La clé de session K_{app} est dérivée via HKDF de $K_{\text{DH}} = g^{ab}$. Pour une session terminée :

1. Les valeurs éphémères a et b sont effacées après la session,
2. Même avec x_U , s , et x_S , $\mathcal{A}$ doit calculer g^{ab} à partir de $A = g^a$ et $B = g^b$,
3. Cela équivaut à résoudre CDH, difficile sous l'hypothèse DDH.

5.3.3 Résistance à la compromission du serveur

Compromission du serveur/server-compromise La compromission de x_S permet à $\mathcal{A}$ de se faire passer pour le serveur, mais ne révèle pas x_U ou s , et ne compromet pas la PFS des sessions établies.

- $\mathcal{A}$ peut signer de faux messages σ_S , mais cela ne lui donne pas accès à K_{DH} sans b ,
- Les secrets client (x_U, s) restent protégés par les preuves ZKP,
- Les sessions passées restent sûres car K_{DH} dépend de a et b éphémères.

5.4 Propriétés de confidentialité

5.4.1 Confidentialité d'identité

Théorème 5.4.1: Confidentialité de l'identité client

Un observateur passif ne peut pas distinguer deux sessions du même client de deux sessions de clients différents.

Construisons un jeu d'indistinguabilité :

1. Le challenger choisit deux identités ID_0, ID_1 ,
2. L'adversaire $\mathcal{A}$ observe les messages de réseau,
3. Le challenger exécute une session avec ID_b où $b \xleftarrow{\$} \{0, 1\}$,
4. $\mathcal{A}$ doit deviner b .

Les messages visibles sont :

- $A = g^a$: uniforme dans G , indépendant de l'identité,
- $B = g^b$: uniforme dans G ,
- auth_C : chiffré avec AEAD sous K_{enc} ,
- $\text{auth}_S, \text{confirm}_C$: également chiffrés.

Sous l'hypothèse DDH, (A, B) sont indistinguables d'éléments aléatoires. Sous IND-CCA d'AEAD, les messages chiffrés ne révèlent pas ID_U . Ainsi, l'avantage de $\mathcal{A}$ est négligeable.

5.4.2 Confidentialité des données de session

Théorème 5.4.2: Confidentialité des clés de session

K_{app} est indistinguable d'une clé aléatoire pour tout adversaire n'ayant pas participé activement à la session.

1. Par DDH, $K_{\text{DH}} = g^{ab}$ est indistinguable d'un élément aléatoire de G ,
2. Par les propriétés d'extracteur de HKDF dans le ROM, $K_{\text{handshake}}$ est uniforme,
3. Les dérivations successives préservent cette uniformité,
4. Ainsi, K_{app} est cryptographiquement forte.

5.5 Résistance aux attaques spécifiques

5.5.1 Attaques par rejeu

Résistance au rejeu replay-resistance Le protocole est résistant aux attaques par rejeu.

Chaque session utilise :

- Un nouveau nonce s généré par le serveur,
- De nouvelles valeurs éphémères a et b ,
- Un identifiant de session $\text{sid} = H(A\|B)$ unique,
- Des timestamps dans les messages chiffrés.

Une tentative de réutiliser un message d'une session précédente échouera car :

1. nonce s différent invalidera le défi c ,
2. La vérification du timestamp détectera les messages trop anciens,
3. Le serveur maintient une liste des sid récents.

5.5.2 Attaques par réflexion

Résistance aux attaques par réflexion reflection-resistance Le protocole prévient les attaques où un adversaire renverrait les messages d'une partie à elle-même.

- Les rôles client/serveur sont asymétriques : le client prouve deux facteurs, le serveur un,
- Les labels de contexte diffèrent : "client-auth" vs "server-finish",
- Les nonces AEAD sont incrémentés différemment (0,1,2),
- Ainsi, un message client ne peut pas être accepté comme message serveur.

5.5.3 Attaques par déni de service (DoS)

Atténuation des DoSdos-mitigation Les mécanismes du protocole atténuent les attaques par déni de service.

1. **Cookie optionnel** : Le serveur peut différer le calcul coûteux de $B = g^b$ jusqu'à vérification d'un cookie HMAC,
2. **Vérification AEAD d'abord** : Le serveur vérifie l'authenticité de auth_C avant les vérifications ZKP coûteuses,
3. **Limites de taux** : Implémentation de limites par IP et par identité,
4. **Timeouts courts** : Sessions expirant rapidement réduisent l'état serveur.

5.5.4 Attaques par canaux auxiliaires

Résistance aux canaux auxiliaires side-channel Le protocole minimise les vecteurs d'attaque par canaux auxiliaires.

- **Temps constant** : Les opérations critiques (vérifications ZKP, AEAD) s'implémentent en temps constant,
- **Comparaison sécurisée** : Utilisation de `constant_time_compare` pour tous les tests,
- **Effacement sécurisé** : Les secrets éphémères (a, b, r_1, r_2) sont effacés après usage,
- **Protection mémoire** : Utilisation de zones mémoire sécurisées pour les clés.

5.6 Limites et hypothèses fortes

5.6.1 Hypothèses cryptographiques

- **DDH dans G** : Nécessite des groupes soigneusement choisis (courbes elliptiques recommandées),
- **Oracle aléatoire** : Bien que pratique, le ROM est un modèle idéalisé,
- **Implémentation correcte** : La sécurité réelle dépend d'une implémentation sans bugs.

5.6.2 Dépendances opérationnelles

- **PKI sécurisée** : Compromission d'une CA briserait l'authentification,
- **Stockage du facteur 2** : s doit être protégé par le dispositif client (SE, TPM),

- **Gestion des clés serveur** : x_S nécessite un HSM ou module matériel sécurisé,
- **Gestion des révocation** : Nécessité de CRL/OCSP à jour.

5.6.3 Limites connues

1. **Absence de preuve UC** : Le protocole n'a pas de preuve formelle dans le modèle de composabilité universelle,
2. **Vulnérabilité au phishing** : Rien n'empêche un faux serveur de capturer les preuves ZKP (mais il ne pourra pas calculer K_{DH} sans b),
3. **Complexité** : 4 exponentiations par partie peut être lourd pour certains dispositifs IoT.

5.7 Comparaison avec les standards existants

Propriété	Sigma-2FA	TLS 1.3	SIGMA-I	IKEv2
Authentification 2FA				
Confidentialité identité				
Forward Secrecy				
Channel binding				
Résistance DoS				
Preuve formelle				

Table 5.1: Comparaison avec les standards (: oui, : partiel, : non)

5.7.1 Avantages sur l'état de l'art

- **Authentification multi-facteur native** : Intégrée au protocole, pas en couche supérieure,
- **Confidentialité d'identité renforcée** : Même le serveur ne connaît l'identité qu'après établissement du canal,
- **Résistance DoS améliorée** : Mécanismes explicites d'atténuation.

5.7.2 Lacunes par rapport à l'état de l'art

- **Complexité cryptographique** : Plus d'opérations que TLS 1.3,
- **Absence d'écosystème** : Pas de support natif dans les navigateurs/OS,
- **Maturité** : Non audité extensivement comme les standards IETF.

5.8 Conclusion de l'analyse

Le protocole Sigma-2FA atteint un niveau de sécurité comparable aux standards industriels modernes tout en fournissant une authentification forte à deux facteurs. Ses propriétés principales sont :

1. **Authentification mutuelle forte** : Basée sur la possession simultanée de deux facteurs,
2. **Confidentialité persistante** : Via Diffie-Hellman éphémère,
3. **Confidentialité d'identité** : Protection contre la surveillance passive,
4. **Résistance aux attaques actives** : MitM, rejeu, réflexion,
5. **Robustesse opérationnelle** : Atténuation DoS, gestion d'erreur sécurisée.

Les limitations principales concernent la complexité cryptographique et l'absence de preuve formelle complète. Pour un déploiement en production, les étapes suivantes sont recommandées :

1. Preuve formelle dans le modèle UC ou sous hypothèses standard,
2. Audit de sécurité par une tierce partie,
3. Implémentation de référence avec tests d'interopérabilité, *intégration progressive avec les infrastructures existantes*.

Perspectives de recherche

- Adapter le protocole pour les dispositifs contraints (IoT),
- Étendre à l'authentification sans mot de passe (WebAuthn),
- Ajouter la post-compromise security (PCS),
- Standardiser comme RFC dans l'IETF.

Chapter 6

Implémentation et validation

6.1 Choix techniques

Cette section doit justifier tous les choix d'implémentation.

6.1.1 Langage

L'étudiant doit choisir un langage favorisant la clarté :

- Python pour le prototypage rapide,
- ou C/Java pour une implémentation plus bas niveau.

6.1.2 Bibliothèques cryptographiques

Lister explicitement :

- gestion des grands entiers,
- opérations modulaires,
- fonctions de hachage sécurisées.

6.1.3 Environnement

Décrire :

- système d'exploitation,
- configuration matérielle,
- hypothèses de déploiement.

6.2 Prototype

Cette section décrit l'implémentation concrète du protocole.

6.2.1 Architecture logicielle

Présenter :

- un module utilisateur,
- un module serveur,
- un module PKI simulé.

6.2.2 Implémentation des ZKP

Décrire précisément :

- génération des engagements,
- calcul des défis,
- vérification des preuves Schnorr [?].

6.2.3 Scénarios de test

Inclure :

- authentification valide,
- tentative sans facteur 1,
- tentative sans facteur 2.

6.3 Résultats expérimentaux

Temps de calcul, taille des messages, observations.

Chapter 7

Implémentation et validation (Exemple)

7.1 Choix techniques

Cette section justifie les décisions d’implémentation pour le prototype du protocole Sigma-2FA.

7.1.1 Langage et paradigmes

Pour concilier sécurité, performance et prototypage, un langage de systèmes offrant un haut niveau d’abstraction sans sacrifier le contrôle est privilégié.

- **Rust** est le choix principal. Sa garantie de sécurité mémoire à la compilation élimine des classes entières de vulnérabilités (déréférencement de pointeur, débordement de buffer). Son modèle de propriété facilite la gestion sécurisée des secrets cryptographiques, qui peuvent être effacés de la mémoire de manière vérifiable. C’est également le langage utilisé dans des travaux récents sur les ZKP non interactifs pour l’IoT [citation:9].
- **Python (optionnel)** : Peut être utilisé pour le développement initial des algorithmes, la simulation de la PKI et l’automatisation des tests d’intégration, en raison de sa rapidité de développement.

7.1.2 Bibliothèques cryptographiques

L’implémentation repose sur des primitives standardisées et auditées.

- **Arithmétique de groupe** : Utilisation de `libsecp256k1` (via `rust-secp256k1`) ou `curve25519-dalek`. Ces bibliothèques fournissent une implémentation sûre et performante des opérations sur courbes elliptiques (multiplication scalaire, additions de points) et gèrent les grands entiers.
- **Fonctions de dérivation et de hachage** : HKDF et SHA-256/512 depuis la crate `rust-crypto` ou `Ring`. La fonction H2S (Hash-to-Scalar) sera implémentée conformément à la RFC 9380.
- **Chiffrement authentifié** : AES-256-GCM ou ChaCha20-Poly1305 via `rust-openssl` ou `chacha20poly1305`.

- **Signatures numériques** : EdDSA (Ed25519) pour les certificats simulés, en cohérence avec le choix d'une courbe elliptique performante.

7.1.3 Environnement d'exécution et hypothèses

- **Système d'exploitation** : Linux (distribution Ubuntu LTS ou Alpine) pour le serveur et le client de test.
- **Matériel de test** :
 - **Serveur** : Machine virtuelle avec **X** cœurs CPU et **Y** Go de RAM.
 - **Client** : Poste de développement standard et émulation d'un dispositif contraint (Raspberry Pi 4) pour évaluer la faisabilité sur terminaux mobiles.
- **Hypothèses de déploiement** :
 - La PKI est simulée par une autorité de certification racine auto-signée émettant des certificats **X.509** pour l'utilisateur et le serveur.
 - Le secret du facteur 2 (s) est stocké dans un "Secure Enclave" logiciel simulé (chiffrement au repos avec une clé dérivée d'un mot de passe).
 - Le réseau est simulé en localhost avec possibilité d'introduire de la latence et des pertes de paquets.

7.2 Architecture du prototype

Cette section décrit l'implémentation concrète et modulaire du protocole.

7.2.1 Architecture logicielle

Le code est organisé en crates Rust (ou modules Python) distincts pour une séparation claire des responsabilités.

- **sigma_2fa_core** : Contient les structures de données, les paramètres cryptographiques, les fonctions de dérivation de clés (HKDF) et les erreurs.
- **sigma_2fa_zkp** : Implémente le cœur cryptographique : la génération des engagements (R_1, R_2), le calcul du défi lié ($c = H_2S(\dots)$), et les réponses (z_1, z_2) de la preuve de Schnorr agrégée [citation:5]. La transformation de Fiat-Shamir est appliquée pour rendre la preuve non interactive côté client [citation:9].
- **sigma_2fa_client** : Module client gérant l'état de la session, la génération des éphémères, la construction des preuves et le chiffrement AEAD des messages d'authentification.

- **sigma_2fa_server** : Module serveur gérant les sessions concurrentes, la vérification des preuves ZKP, la signature de l'authentification et la confirmation de clé.
- **sigma_2fa_pki** (simulé) : Gère la génération, la signature et la vérification de certificats racine et utilisateur pour les besoins du test.
- **sigma_2fa_sim** : Programme principal orchestrant les scénarios de test et collectant les métriques.

7.2.2 Implémentation détaillée des ZKP

L'implémentation de la preuve de Schnorr agrégée suit rigoureusement les étapes théoriques.

1. **Génération des engagements** : Pour un secret x (ou s) et un générateur g , l'algorithme choisit un aléa $r \xleftarrow{\$} \mathbb{Z}_q$ et calcule $R = g^r$ en utilisant la multiplication scalaire de la courbe elliptique.
2. **Calcul du défi lié** : Le défi c n'est pas choisi aléatoirement mais déterministiquement via $c = \text{H2S}(A \parallel B \parallel \text{nonces}_S \parallel R_1 \parallel R_2 \parallel \text{"challenge"})$. Cette étape lie la preuve à la session et rend l'attaque par rejeu impossible.
3. **Calcul et vérification des réponses** : La réponse est calculée comme $z = r + c \cdot x \pmod q$. La vérification s'effectue en testant l'égalité $g^z \stackrel{?}{=} R \cdot Y^c$, où $Y = g^x$ est la clé publique connue. Cette étape est au cœur du "Schéma d'Identification de Schnorr" [citation:5][citation:9].

Placeholder Figure 1 : Diagramme de séquence UML illustrant l'interaction entre les modules lors d'une authentification réussie.

7.2.3 Scénarios de test et validation

Les tests visent à valider le bon fonctionnement et la robustesse aux erreurs.

- **Scénario nominal** : Authentification complète avec les deux facteurs valides. Vérification que la clé de session K_{app} est identique des deux côtés.
- **Tentative sans le facteur 1** : Le client tente de s'authentifier avec seulement le secret s (facteur 2). Le serveur doit rejeter la connexion lors de la vérification de z_1 .
- **Tentative sans le facteur 2** : Le client tente de s'authentifier avec seulement le secret x_U (facteur 1). Le serveur doit rejeter la connexion lors de la vérification de z_2 .
- **Attaque par rejeu** : Réinjection d'un message `auth_C` d'une session précédente. Le serveur doit le détecter grâce à l'unicité de nonces_S et/ou au *channel binding* dans le défi c .

- **Altération de message** : Modification d'un bit dans le message B ou auth_C pendant la transmission. Doit conduire à un échec de déchiffrement AEAD ou de vérification de la signature finale.

7.3 Résultats expérimentaux et analyse

Cette section présente les métriques objectives du prototype, à mesurer et à reporter dans les tableaux ci-dessous.

7.3.1 Méthodologie de mesure

- **Outils** : Utilisation de la crate `criterion.rs` pour le benchmarking en Rust, et de scripts Python pour l'analyse statistique.
- **Métriques** : Temps de calcul (moyenne, écart-type sur **N=1000** exécutions), taille des messages échangés (en octets), et consommation mémoire.
- **Conditions** : Toutes les mesures sont effectuées sur le matériel de test spécifié, en mode *release* avec optimisations activées.

7.3.2 Performances cryptographiques

Le tableau suivant présente les temps d'exécution des opérations les plus coûteuses.

Placeholder Tableau 1 : Temps de calcul des opérations cryptographiques (en millisecondes).

Opération	Côté Client
Génération de la paire éphémère (DH)	**Valeur** ms
Calcul de la preuve ZKP agrégée (z_1, z_2)	**Valeur** ms
Vérification de la preuve ZKP agrégée	–
Signature du transcript (serveur)	–
Dérivation complète des clés	**Valeur** ms
Temps total d'établissement de session	**Valeur**

*Note : Les valeurs sont à remplacer après benchmarking. Le "temps total" inclut la communication réseau simulée (latence fixe de **1** ms).*

7.3.3 Analyse de l'overhead de communication

Le protocole échange des messages dont la taille est fixe et déterministe, un avantage pour les réseaux à bande passante limitée.

Placeholder Tableau 2 : Taille des messages échangés (en octets).

Message	Taille (Théorique)	Taille (Mesurée)
$A = g^a$ (clé éphémère)	$\text{Taille}(A) = 32$ (compressed)	$\text{Taille}(A) = 32$
B , nonces, sid	$32 + 16 + 32 = 80$	$\text{Taille}(B) = 80$
auth_C (AEAD)	$\text{Taille}(\text{ID}_U, R_1, R_2, z_1, z_2) + \text{Overhead AEAD}$	$\text{Taille}(\text{auth}_C) = \text{Taille}(\text{ID}_U, R_1, R_2, z_1, z_2) + \text{Overhead AEAD}$
auth_S (AEAD)	$\text{Taille}(\sigma_S, \text{fin}_S) + \text{Overhead AEAD}$	$\text{Taille}(\text{auth}_S) = \text{Taille}(\sigma_S, \text{fin}_S) + \text{Overhead AEAD}$
confirm_C (AEAD)	$\text{Taille}(\text{fin}_C) + \text{Overhead AEAD}$	$\text{Taille}(\text{confirm}_C) = \text{Taille}(\text{fin}_C) + \text{Overhead AEAD}$
Total octets échangés		Valeur octets

7.3.4 Validation des propriétés de sécurité

Au-delà des tests fonctionnels, des analyses spécifiques sont nécessaires.

- **Analyse de fuite d'information** : Utilisation d'outils comme `valgrind` ou de l'`AddressSanitizer` de Rust pour s'assurer que les secrets (clés privées, nonces) ne sont pas écrits en clair dans la mémoire ou les logs. Vérification que les tampons contenant des secrets sont explicitement effacés (`zeroize`).
- **Résistance temporelle** : Vérification que le temps d'exécution des opérations de vérification (égalité de points, vérification ZKP) est constant, quelle que soit l'entrée, pour éviter les attaques par canaux auxiliaires.
- **Analyse formelle partielle** : Modélisation des étapes clés du protocole (échange DH, ZKP agrégée) dans un outil comme `Tamarin Prover` [citation:9] ou `ProVerif` pour vérifier automatiquement des propriétés comme le secret de session et l'accord injectif. *(Cette étape dépasse le cadre d'un prototype mais doit être mentionnée comme perspective).*

Placeholder Figure 2 : Graphique comparant le temps d'établissement de session du protocole Sigma-2FA avec un handshake TLS 1.3 (ECDHE-ECDSA) dans des conditions similaires.

7.3.5 Limitations du prototype et perspectives

- **Performance réelle en réseau** : La latence et les pertes de paquets sur un réseau étendu (WAN) peuvent affecter le temps total perçu, bien que le nombre de tours (2-3) soit compétitif.
- **Sécurité matérielle** : Ce prototype logiciel ne bénéficie pas des protections d'un véritable élément sécurisé (SE) ou d'un module de plateforme de confiance (TPM) pour stocker s .
- **Évolutivité serveur** : La vérification des preuves ZKP, bien que rapide, doit être profilée sous une charge concurrente élevée pour évaluer la nécessité d'optimisations (e.g., traitement par lots).

Conclusion du chapitre

Ce chapitre a défini le cadre méthodologique complet pour l'implémentation et l'évaluation du protocole Sigma-2FA. Les choix techniques (Rust, bibliothèques cryptographiques standardisées) et l'architecture modulaire posent les bases d'un prototype robuste et mesurable. Les tableaux et figures préparés permettront de documenter de manière systématique les performances (temps de calcul, overhead réseau) et de valider empiriquement les propriétés de sécurité du protocole une fois le développement et les tests effectués. La prochaine étape concrète consiste à implémenter

les modules décrits et à exécuter la campagne de benchmarks pour remplacer les placeholders `<span style="color:red;">**en rouge**</span>` par des résultats tangibles.

Chapter 8

Conclusion et perspectives

8.1 Bilan

Récapituler les apports réels du travail.

8.2 Perspectives

Extensions possibles :

- preuves formelles,
- intégration applicative,
- optimisation.

Définition de la version article dérivée

L'article dérivé du présent mémoire ne constitue pas une reprise intégrale du document.

Périmètre de l'article

L'article se concentre exclusivement sur :

- la description formelle du protocole,
- le modèle de menace,
- l'analyse de sécurité,
- une discussion comparative qualitative.

Éléments exclus

- revue pédagogique détaillée,
- implémentation complète,
- aspects purement didactiques.

Titre indicatif de l'article

A Diffie-Hellman-Based Zero-Knowledge Two-Factor Authentication Protocol for Hierarchical PKI Environments

Positionnement scientifique

L'article est positionné comme une contribution protocolaire exploratoire, sans prétention à une standardisation immédiate.

Bibliography

- [1] W. Diffie and M. Hellman, “New Directions in Cryptography,” *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976. doi: 10.1109/TIT.1976.1055638.
- [2] C.-P. Schnorr, “Efficient Signature Generation by Smart Cards,” *Journal of Cryptology*, vol. 4, no. 1, pp. 161–174, 1991. doi: 10.1007/BF00196725.
- [3] D. Chaum and T. P. Pedersen, “Wallet Databases with Observers,” in *Advances in Cryptology – CRYPTO ’92*, LNCS, vol. 740, Springer, 1993, pp. 89–105.
- [4] O. Goldreich, *Foundations of Cryptography: Volume 1, Basic Tools*, Cambridge University Press, 2001.
- [5] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 2nd ed., CRC Press, 2014.
- [6] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, Version 0.6, 2023. [Online]. Available: <http://toc.cryptobook.us/>
- [7] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, IETF, RFC 8446, 2018.
- [8] M. Bellare and P. Rogaway, “Entity Authentication and Key Distribution,” in *Advances in Cryptology – CRYPTO ’93*, LNCS, vol. 773, Springer, 1994, pp. 232–249.
- [9] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.
- [10] P. A. Grassi et al., *Digital Identity Guidelines*, NIST Special Publication 800-63-3, National Institute of Standards and Technology, 2017. doi: 10.6028/NIST.SP.800-63-3.